11 December 2024

EFGs: A Complete and Accurate Implementation of Ertl's Functional Group Detection Algorithm in RDKit

Gonzalo Colmenarejo[1]

1. Biostatistics & Bioinformatics Unit, IMDEA Food

Abstract

Functional groups are widely used in Organic Chemistry, as they provide a rationale to analyze physicochemical and reactivity properties. In Medicinal Chemistry they are the basis for analyzing ligand-biomacromolecule interactions. Ertl's algorithm is an approach to extract functional groups in arbitrary organic molecules that do not depend on predefined libraries of functional groups. However, there is a lack of a complete and accurate implementation of Ertl's algorithm in the widely used RDKit cheminformatic toolkit. In this paper a new RDKit/Python implementation of the algorithm is described, that is both accurate and complete. For a RDKit molecule, it provides: a) a PNG binary string with an image of the molecule with color-highlighted functional groups; b) a list of sets of atom indices (idx), each set corresponding to a functional group; c) a list of pseudo-SMILES canonicalized strings for the full functional groups; d) a list of RDKit labeled mol objects, one for each full functional group. The code is freely available in github.com/bbu-imdea/efgs

Keywords

Ertl algorithm, functional group, drug discovery, cheminformatics, ligand-receptor interactions

Posted on 11 December 2024 — CC-BY-NC-ND 4.0 — This is a preprint and has not been peer reviewed. Data may be preliminary. —
https://doi.org/10.26434/chemrxiv-2024-ptsbg

EFGs: A Complete and Accurate Implementation of Ertl's Functional Group Detection Algorithm in RDKit

Gonzalo Colmenarejo*

Biostatistics and Bioinformatics Unit

IMDEA Food

CEI UAM+CSIC

E28049 Madrid, Spain

*Corresponding Author

e-mail: gonzalo.colmenarejo@imdea.org

https://doi.org/10.26434/chemrxiv-2024-ptsbg **ORCID:** https://orcid.org/0000-0002-8249-4547 Content not peer-reviewed by ChemRxiv. **License:** CC BY-NC-ND 4.0

ABSTRACT

Functional groups are widely used in Organic Chemistry, as they provide a rationale to analyze physicochemical and reactivity properties. In Medicinal Chemistry they are the basis for analyzing ligand-biomacromolecule interactions. Ertl's algorithm is an approach to extract functional groups in arbitrary organic molecules that do not depend on predefined libraries of functional groups. However, there is a lack of a complete and accurate implementation of Ertl's algorithm in the widely used RDKit cheminformatic toolkit. In this paper a new RDKit/Python implementation of the algorithm is described, that is both accurate and complete. For a RDKit molecule, it provides: a) a PNG binary string with an image of the molecule with color-highlighted functional groups; b) a list of sets of atom indices (idx), each set corresponding to a functional group; c) a list of pseudo-SMILES canonicalized strings for the full functional groups; d) a list of RDKit labeled mol objects, one for each full functional group. The code is freely available in https://github.com/bbu-imdea/efgs

KEYWORDS:

Ertl algorithm, functional group, drug discovery, cheminformatics, ligand-receptor interactions

https://doi.org/10.26434/chemrxiv-2024-ptsbg **ORCID:** https://orcid.org/0000-0002-8249-4547 Content not peer-reviewed by ChemRxiv. **License:** CC BY-NC-ND 4.0

INTRODUCTION

The concept of functional groups (FGs), defined as sets of connected atoms in a molecule with specific physicochemical and reactivity properties, is central in Organic and Medicinal Chemistry.[1] On one hand, it allows to rationalize and predict the set of chemical reactions amenable for a compound, as well as its physical properties. On the other hand, it has a large explanatory power to model and understand ligand-protein and ligand-nucleic acid interactions.[2,3]

Although there are different cheminformatic approaches to analyze FGs in molecules, they are typically based on predefined libraries of FGs.[4,5] In this regard, an alternative approach was developed by Peter Ertl, who proposed an algorithm to detect arbitrary FGs in any molecule.[6] The power of this approach lies in not being based on predefined FGs, but instead in being able to deal with sets of organic molecules of arbitrary origins and chemistries. As a result, the approach has been widely applied to drugs,[6,7] natural products,[8] and microbial metabolites.[9]

Ertl implemented his algorithm in a Novartis-proprietary cheminformatic toolkit, inaccessible to the wide Cheminformatics community.[6] However, since the algorithm details were provided, soon two freely available implementations appeared, the first (*IFG*)[10] in the RDKit/Phython toolkit[11], and the second (*ErtlFunctionalGroupsFinder*)[12] in CDK/Java.[13,14]

All in all, when attempting a FG analysis of microbial metabolites in our recent publication,[9] it was possible to realize that the RDKit implementation had a series of

https://doi.org/10.26434/chemrxiv-2024-ptsbg **ORCID:** https://orcid.org/0000-0002-8249-4547 Content not peer-reviewed by ChemRxiv. **License:** CC BY-NC-ND 4.0

issues, resulting in not reproducing Ertl's algorithm correctly (see below). In addition, the IFG implementation does not generate pseudo-SMILES with the generalized FGs that Ertl's algorithm provides, which include special substrings ([R], [Nar], [Oar], [Car], [Cal], etc.). Finally, the code neither provide graphical representation of the FGs nor of the FG-highlighted full molecules.

Thus, a new code was written in Python, using the RDKit toolkit, to provide an accurate and complete implementation of Ertl's algorithm. Since the code developed could be useful for the whole Cheminformatics community, here it is made publicly available. In this work, the new implementation, EFGs (Ertl's FG's), is described in detail, and its use is showcased in an analysis of FGs in the whole set of bioactive molecules in ChEMBL33.[15,16]

https://doi.org/10.26434/chemrxiv-2024-ptsbg **ORCID:** https://orcid.org/0000-0002-8249-4547 Content not peer-reviewed by ChemRxiv. **License:** CC BY-NC-ND 4.0

IMPLEMENTATION

Ertl's algorithm

Ertl's algorithm generates pseudo-SMILES strings that represent both the FG itself and the "environment" (directly attached) carbons and hydrogens. Although they follow SMILES[17,18] syntax, they include "atypical" substrings (([R], [Nar], [Oar], [Car], [Cal], etc.) that represent atoms or sets of alternative atoms with specific features.

Ertl's FG algorithm[6] is here described succinctly:

1. Initial atom marking: all nonaromatic heteroatoms (including halogens) are marked, plus the following carbon atoms:
    a. Carbon atoms connected to any heteroatom through double / triple nonaromatic bonds.
    b. Pairs of carbon atoms in doble or triple nonaromatic C-C bonds.
    c. Carbon atoms in acetal-like groups, i.e. sp3 carbons connected to two or more atoms of oxygen, nitrogen, or sulfur (pairs with mixed elements, e.g. one O and one N, are included here); in addition, these heteroatoms must only have single bonds.
    d. Carbon atoms in oxirane, aziridine, and thiirane rings
2. Atom merging: once all these atoms are marked, FGs are obtained by merging all the connected atoms (but see exception in 3 below). The result is for each molecule a set of disjoint graphs.

https://doi.org/10.26434/chemrxiv-2024-ptsbg **ORCID:** https://orcid.org/0000-0002-8249-4547 Content not peer-reviewed by ChemRxiv. **License:** CC BY-NC-ND 4.0

3. Aromatic heteroatoms are marked as single-atom FGs, even if they are in adjacent positions in a ring (adjacent ones are not merged into a single FG as in 2). They are represented in the pseudo-SMILES as [Nar], [Oar], and [Sar], respectively for N, O, and S. They are only extended if they are adjacent to a FG through a nonaromatic connection.
4. Generalization of FGs ("decorated" FGs): after extracting the FG atoms, their adjacent, environment atoms (Hs or Cs) are represented as follows:
    a. If the environment atom is a carbon bonded to a carbon in the FG, it is not included, except in the case of carbonyl carbons (in order to distinguish between aldehydes and ketones). In this case they are represented by [R] in the pseudo-SMILES.
    b. The remaining free valences in the heteroatoms of the FG are now filled with hydrogen or with aliphatic or aromatic carbon atoms. These environment atoms are represented as follows:
        i. Hydrogens are graphically represented as such in -OH groups, simple amines and thiols, although they are not included in the pseudo-SMILES (implicit hydrogens). In this way, secondary and tertiary amines are distinguished, and the same for alcohols vs ethers, and thiols vs sulfides.
        ii. The remaining environment carbon atoms are replaced by [R]; however, if the FG is a single atomic N or O, they are replaced by [Cal] in the case of aliphatic carbons, and by [Car] in the case of

https://doi.org/10.26434/chemrxiv-2024-ptsbg **ORCID:** https://orcid.org/0000-0002-8249-4547 Content not peer-reviewed by ChemRxiv. **License:** CC BY-NC-ND 4.0

aromatic ones. This permits the distinction between amines and anilines, and between alcohols and phenols.

The substrings [R], [Cal], [Car], [Nar], [Oar], and [Sar] in the pseudo-SMILES are shown graphically as R, Cal, Car, Nar, Oar, and Sar, respectively.

RDKit computational details

The implementation in Python/RDKit of Ertl's algorithm here provided uses functionality from the toolkit to handle the specific requirements of the complete algorithm. The main function to generate "decorated" FGs, that is, with environment substrings, is called *get_dec_fgs*, that uses as sole input a RDKit mol object. For an input molecule, the function returns four outputs: a) a PNG binary string with an image of the molecule having its FG-belonging atoms highlighted in colors by type of FG, like in Figure 1 of Ertl's paper;[6] the binary string is generated form a rdMolDraw2D.MolDraw2DCairo object; b) a list of sets of atom indices (idx), each set corresponding to a FG; c) a list of pseudo-SMILES strings for the decorated FGs, as in Supplementary Information of Ertl's paper,[6] following the same order as the list of idx's, to be able to map de pseudo-SMILES to the corresponding idx's; and d) a list of RDKit labeled mol objects, one for each decorated FG and in the same order as the other lists (again, to facilitate the mapping with idx's), as in Figures 3 and 4 of Ertl's paper.[6]

The *get_dec_fgs* function starts by removing all the chiral and stereo information from atoms and bonds in the molecule, and then making all hydrogen atoms explicit in the molecule (Figure 1). Then it retrieves: a) the idx's of the non-decorated FGs, as in steps

https://doi.org/10.26434/chemrxiv-2024-ptsbg **ORCID:** https://orcid.org/0000-0002-8249-4547 Content not peer-reviewed by ChemRxiv. **License:** CC BY-NC-ND 4.0

1, 2, 3 of Ert's algorithm above (resulting in the second output of the function); b) the environment atoms bond to carbonyl carbon atoms to be used in 4.a; c) the free valence atoms bound to heteroatoms to be used in 4.b; d) the environment carbon atom idx's for single atomic N or O FGs, together with information about aromaticity, to be used in 4.b.ii; and f) the aromatic carbons double bonded to FGs. The identification of all these atoms is performed mainly through SMARTS strings *ad hoc* designed for each case, plus analysis of the atom elements and bond types.

After that, the hydrogen atoms that do not to appear explicitly in the decorated FGs are transformed into dummy atoms, and the molecule is returned to its implicit-hydrogen version, with an update of the idx to deal with the now missing dummy atoms.

With all this information, the function iterates over each FG, generating a fragment molecule with the appropriate labels for environment atoms (R, Cal, Car, Nar, etc.). These fragment molecules comprise the fourth output of the function.

In order to generate the pseudo-SMILES, an initial string is generated by applying the Chem.MolToSmiles function of RDKit into the previous fragment molecules. Then some fixes are applied to the strings to represent appropriately the labels as substrings, and then a final canonicalization function is applied to generate a canonical pseudo-SMILES. This is the third output of the function.

Finally, the idx's of the FGs identified above are applied to a copy previously generated of the original molecule in order to generate a rdMolDraw2D.MolDraw2DCairo version

https://doi.org/10.26434/chemrxiv-2024-ptsbg **ORCID:** https://orcid.org/0000-0002-8249-4547 Content not peer-reviewed by ChemRxiv. **License:** CC BY-NC-ND 4.0

of the molecule with highlighted FGs in colors depending on the type of FG. From here a binary PNG string is generated, resulting in the first output of the function.

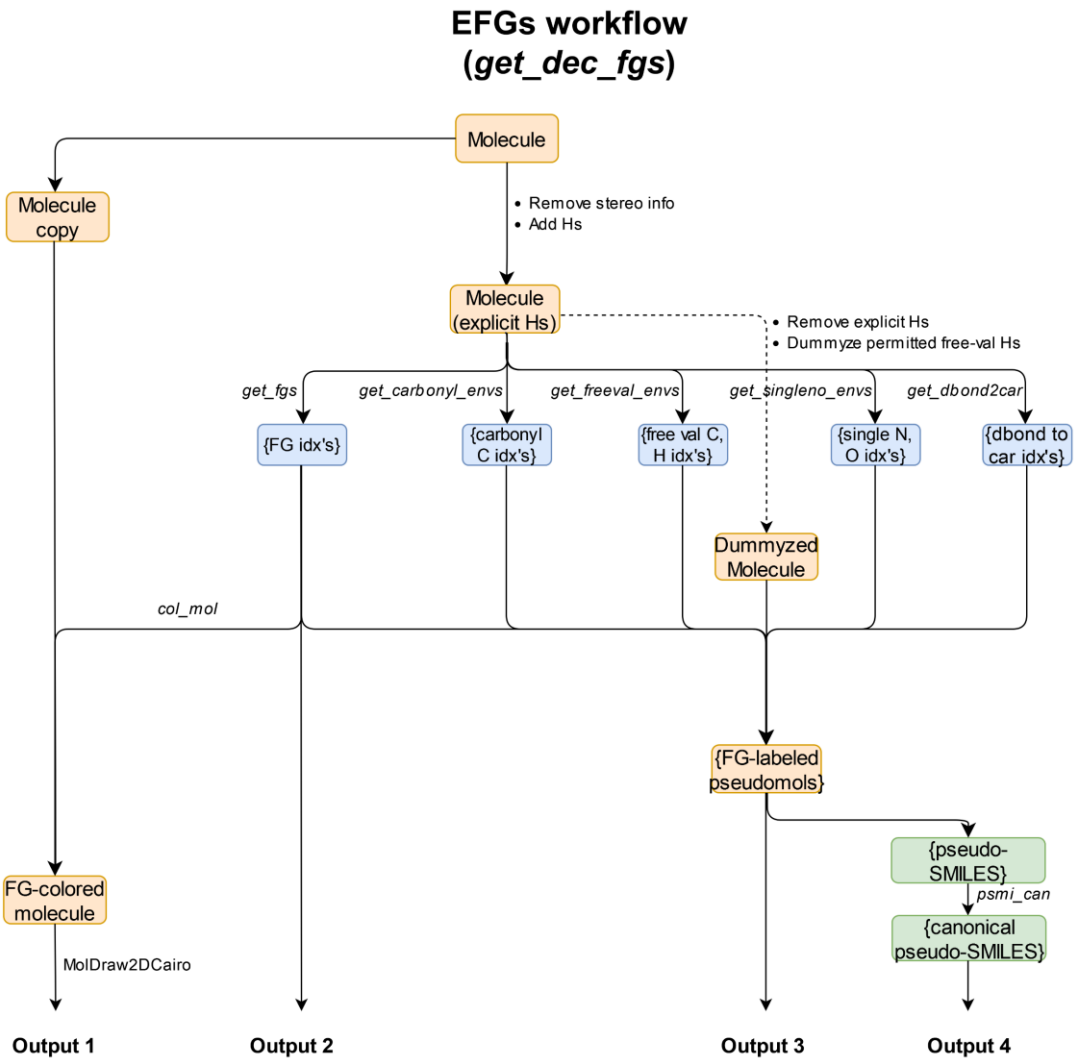


Figure 1. *Workflow of the EFGs implementation of Ertl's algorithm in RDKit. The name of the different functions applied at different stages is shown in italics.*

Since the *get_dec_fgs* function only needs a RDKit molecule object, in can be easily vectorized with the Pandas apply function if applied to a column of molecule objects in a dataframe or a series. Similarly, it can be parallelized in a straightforward way by

https://doi.org/10.26434/chemrxiv-2024-ptsbg **ORCID:** https://orcid.org/0000-0002-8249-4547 Content not peer-reviewed by ChemRxiv. **License:** CC BY-NC-ND 4.0

using Pandas parallelization modules: MultiprocessPandas, Modin, Pandarallel, etc. The functions used only require RDKit and the re Python module, the later being part of the Python standard package.

The code can be freely obtained from the https://github.com/bbu-imdea/efgs repository. There main functionality is in the *efgs.py* script. In addition, a *try_efgs.py* script is provided to exemplify the use of the vectorization of the FG extraction. In that script are additional utility functions are also included to generate high-resolution grid images of sets of FGs and highlighted molecules, draw images of highlighted molecules from binary PING strings, and display the molecules with idx labels.

https://doi.org/10.26434/chemrxiv-2024-ptsbg **ORCID:** https://orcid.org/0000-0002-8249-4547 Content not peer-reviewed by ChemRxiv. **License:** CC BY-NC-ND 4.0

RESULTS

As described above, the application of the IFG code in RDKit/contrib to the analysis of microbial metabolites recently conducted by us[9] allowed to identify issues in this implementation of Ertl's algorithm. Namely: a) adjacent aromatic heteroatoms are merged in the same FG; this is not expected in Ertl's algorithm, where each heteroaromatic atom forms a single FG, which is only extended if it is bonded through a nonaromatic bond to another FG; b) the output SMILES strings are not fully canonicalized; c) the SMILES string of the generalized FG is generated by simply adding the adjacent carbon atoms, "c" if it is aromatic, and "C" if it is aliphatic, irrespective of the nature of the atoms they are connected to within the FG, such that rules 4.a. and 4.b. above are not followed; d) the SMILES strings include stereo information. All these factors result in a multiplication of the FG SMILES strings.

In order to see the functionalities of EFGs, it was applied to a large set of bioactive compounds, namely, the compounds in ChEMBL33 with a pChEMBL above 5 for an interaction with a protein of type "PROTEIN" or "PROTEIN COMPLEX". After standardization and parentization with the ChEMBL Structure Pipeline[19] functions, this resulted in a total of 839063 unique molecules. Standardization and especially parentization are very important requirements, since a) Ertl's algorithm is defined for a single molecule, and b) in many cases the non-parent molecules (salts, etc.) have atoms with elements outside the range of the typical ones in organic chemistry (C, H, N, O, X, S, P, Si, Se, Te), like alkali and transition metals. As a result, applying the code

https://doi.org/10.26434/chemrxiv-2024-ptsbg **ORCID:** https://orcid.org/0000-0002-8249-4547 Content not peer-reviewed by ChemRxiv. **License:** CC BY-NC-ND 4.0

to non-parentized and non-organic molecules can fail to provide all the FGs and/or the FG-colored image of the molecule, as they are outside the scope of the algorithm.

From the ChEMBL set above described a total of 5836 unique FGs were identified. Instead, the application of IFG code gave a total of 14480 different FGs. The much larger number of FGs in the later can be ascribed to the lack of generalization as required by rules in item 4 above, plus the presence of FGs obtained by merging adjacent aromatic heteroatoms, not allowed by rule 3 above. Figure 2 displays the top 48 most abundant FGs in this set of molecules obtained with EFGs, in decreasing order. This is equivalent to Figure 3 of Ert's paper,[6] but now both the images of the FGs and the pseudo-SMILES are generated with EFGs (outputs 3 and 4, respectively).

https://doi.org/10.26434/chemrxiv-2024-ptsbg **ORCID:** https://orcid.org/0000-0002-8249-4547 Content not peer-reviewed by ChemRxiv. **License:** CC BY-NC-ND 4.0

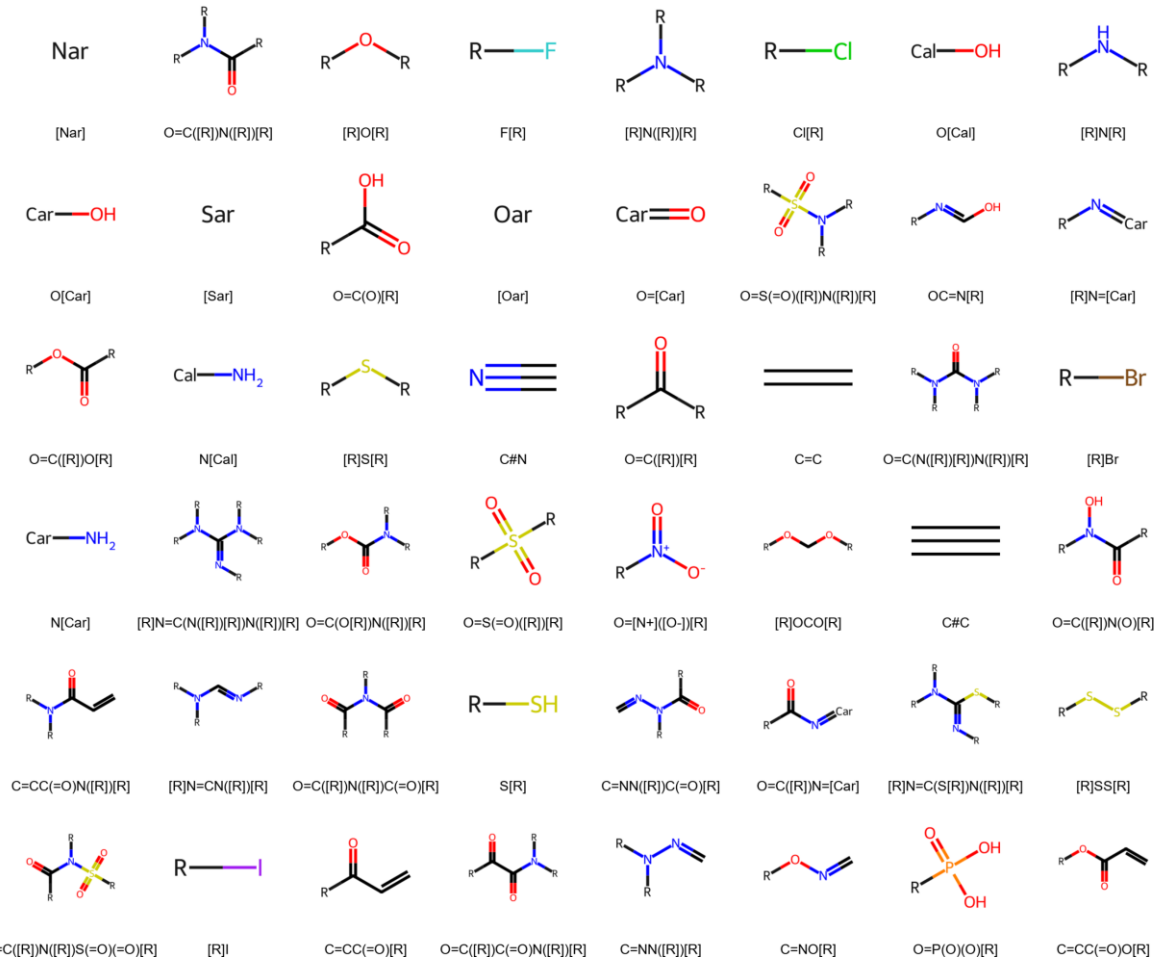
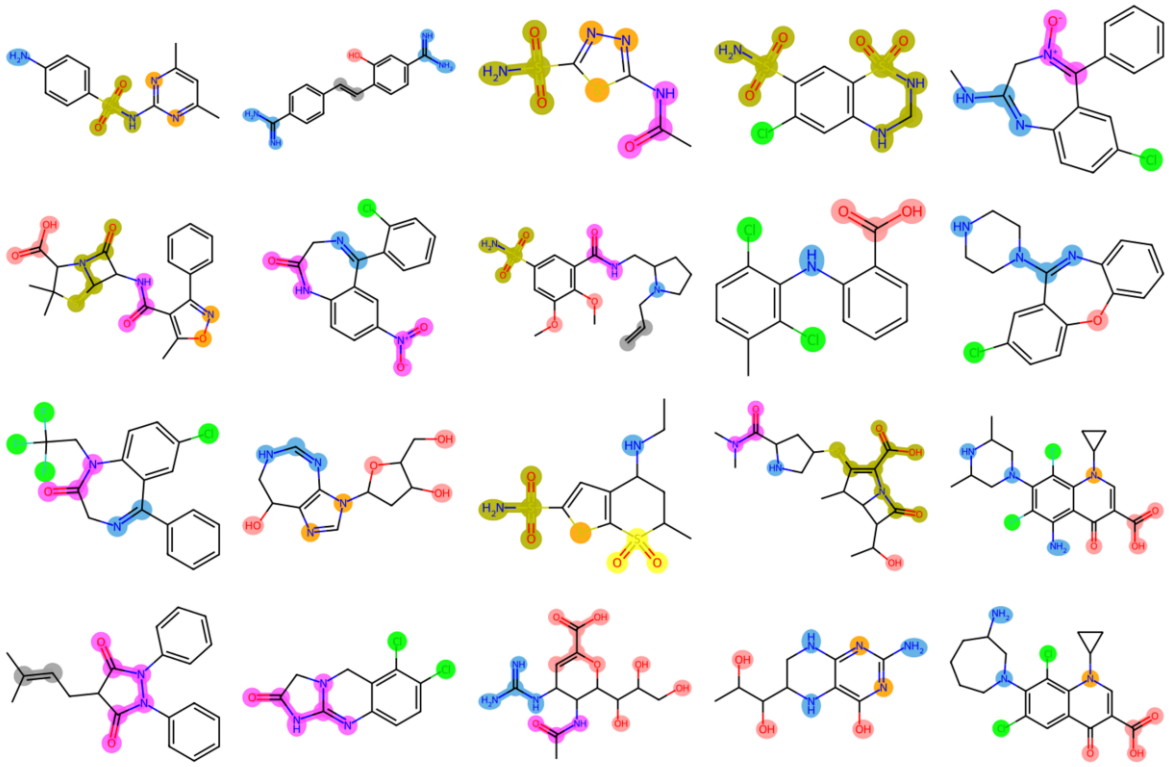


Figure 2. *Top 48 FGs from ChEMBL33 bioactive compounds (equivalent to Figure 2 in Ert's paper[6]). To generate the images of the FGs, the RDKit labeled molecules generated with EFGs (output 3) have been used. The strings under each image are the FG canonical pseudo-SMILES generated with EFGs (output 4). FGs are shown in decreasing order, from left to right and from top to bottom rows.*

In spite of the two figures being generated with a near 10 years difference in the versions of ChEMBL, the two distributions are quite similar, indicating a marked conservation in chemotypes across time.

https://doi.org/10.26434/chemrxiv-2024-ptsbg **ORCID:** https://orcid.org/0000-0002-8249-4547 Content not peer-reviewed by ChemRxiv. **License:** CC BY-NC-ND 4.0

Figure 3 displays FG-highlighted molecules obtained with EFGs (output 1) when applied to the molecules appearing in Figure 1 of Ertl's paper.[6] The images are generated in PNG format by applying pilImage.open(io.BytesIO(x)), where x is the highlighted molecule PNG binary string. The same FGs are identified in the two figures; a single discrepancy in the 10th molecule, the last molecule in the second row, is due to RDKit not considering aromatic the central oxazepine ring in the molecule, while Ertl's toolkit did. As a consequence, here the nitrogen in that ring is merged with the closest nitrogen in the piperazine ring in a shared FG, while in Ertl's figure it remains as a single FG due to rule 3 above. In other words, the EFGS implementation of the algorithm is correct, and the discrepancies arise from the different aromaticity perceptions of the toolkits.



https://doi.org/10.26434/chemrxiv-2024-ptsbg **ORCID:** https://orcid.org/0000-0002-8249-4547 Content not peer-reviewed by ChemRxiv. **License:** CC BY-NC-ND 4.0

Figure 3. *FG-highlighted molecules using EFGs/RDKit. The molecules are the same as in Figure 1 in Ert's paper.*[6] *A single discrepancy is observed for molecule 10 (last molecule in the second row). This is because the toolkit used by Ert considers the central oxazepine ring to be aromatic, and so its nitrogen atom, that would form a single-atom FG. In contrast, RDKit does not consider that atom to be aromatic and merges it with the nearest piperazine nitrogen.*

https://doi.org/10.26434/chemrxiv-2024-ptsbg **ORCID:** https://orcid.org/0000-0002-8249-4547 Content not peer-reviewed by ChemRxiv. **License:** CC BY-NC-ND 4.0

CONCLUSIONS

The EFGs code provides a complete and accurate implementation of Ertl's algorithm for the RDKit/Python Cheminformatics community. It is expected to be useful for any type of organic molecules as a tool to analyze FGs. The code is freely available in https://github.com/bbu-imdea/efgs.

https://doi.org/10.26434/chemrxiv-2024-ptsbg **ORCID:** https://orcid.org/0000-0002-8249-4547 Content not peer-reviewed by ChemRxiv. **License:** CC BY-NC-ND 4.0

AUTHOR INFORMATION

Corresponding Author:

Gonzalo Colmenarejo - Biostatistics and Bioinformatics Unit. IMDEA Food, CEI UAM+CSIC, E28049 Madrid, Spain. https://orcid.org/0000-0002-8249-4547.

Notes

The author declare no competing financial interest.

DATA AND SOFTWARE AVAILABILITY STATEMENT

The code of EFGs is freely available in https://github.com/bbu-imdea/efgs.

SUPPORTING INFORMATION

No Supporting Information is provided.

https://doi.org/10.26434/chemrxiv-2024-ptsbg **ORCID:** https://orcid.org/0000-0002-8249-4547 Content not peer-reviewed by ChemRxiv. **License:** CC BY-NC-ND 4.0

REFERENCES

(1) Hanson, J. R.; Hanson, J. R. *Functional Group Chemistry*; Tutorial chemistry texts; Royal Society of Chemistry: Cambridge, 2001.

(2) Mukherjee, G.; Braka, A.; Wu, S. Quantifying Functional-Group-like Structural Fragments in Molecules and Its Applications in Drug Design. *J. Chem. Inf. Model.* **2023**, *63* (7), 2073–2083. https://doi.org/10.1021/acs.jcim.3c00050.

(3) Yang, L.; He, W.; Yun, Y.; Gao, Y.; Zhu, Z.; Teng, M.; Liang, Z.; Niu, L. Defining A Global Map of Functional Group-Based 3D Ligand-Binding Motifs. *Genomics, Proteomics & Bioinformatics* **2022**, *20* (4), 765–779. https://doi.org/10.1016/j.gpb.2021.08.014.

(4) *Checkmol/Matchmol Homepage*. https://homepage.univie.ac.at/norbert.haider/cheminf/cmmm.html (accessed 2024-10-31).

(5) Salmina, E. S.; Haider, N.; Tetko, I. V. Extended Functional Groups (EFG): An Efficient Set for Chemical Characterization and Structure-Activity Relationship Studies of Chemical Compounds. *Molecules* **2016**, *21* (1), 1. https://doi.org/10.3390/molecules21010001.

(6) Ertl, P. An Algorithm to Identify Functional Groups in Organic Molecules. *Journal of Cheminformatics* **2017**, *9* (1), 36. https://doi.org/10.1186/s13321-017-0225-z.

(7) Ertl, P.; Altmann, E.; McKenna, J. M. The Most Common Functional Groups in Bioactive Molecules and How Their Popularity Has Evolved over Time. *J. Med. Chem.* **2020**, *63* (15), 8408–8418. https://doi.org/10.1021/acs.jmedchem.0c00754.

(8) Ertl, P.; Schuhmann, T. A Systematic Cheminformatics Analysis of Functional Groups Occurring in Natural Products. *J. Nat. Prod.* **2019**, *82* (5), 1258–1263. https://doi.org/10.1021/acs.jnatprod.8b01022.

(9) Astigarraga, M.; Sánchez-Ruiz, A.; Diop-Aw, A.; Quintero, R.; Colmenarejo, G. How Do Microbial Metabolites Interact with Their Protein Targets? ChemRxiv October 15, 2024. https://doi.org/10.26434/chemrxiv-2024-q30x2.

(10) *rdkit/Contrib/IFG at master · rdkit/rdkit · GitHub*. https://github.com/rdkit/rdkit/tree/master/Contrib/IFG (accessed 2024-10-31).

(11) *RDKit: Open-source cheminformatics*. https://www.rdkit.org/ (accessed 2021-09-03).

(12) Fritsch, S.; Neumann, S.; Schaub, J.; Steinbeck, C.; Zielesny, A. ErtlFunctionalGroupsFinder: Automated Rule-Based Functional Group Detection with the Chemistry Development Kit (CDK). *Journal of Cheminformatics* **2019**, *11* (1), 37. https://doi.org/10.1186/s13321-019-0361-8.

(13) Steinbeck, C.; Han, Y.; Kuhn, S.; Horlacher, O.; Luttmann, E.; Willighagen, E. The Chemistry Development Kit (CDK): An Open-Source Java Library for Chemo- and Bioinformatics. *J. Chem. Inf. Comput. Sci.* **2003**, *43* (2), 493–500. https://doi.org/10.1021/ci025584y.

(14) Willighagen, E. L.; Mayfield, J. W.; Alvarsson, J.; Berg, A.; Carlsson, L.; Jeliazkova, N.; Kuhn, S.; Pluskal, T.; Rojas-Chertó, M.; Spjuth, O.; Torrance, G.; Evelo, C. T.; Guha, R.; Steinbeck, C. The Chemistry Development Kit (CDK) v2.0: Atom Typing, Depiction,

https://doi.org/10.26434/chemrxiv-2024-ptsbg **ORCID:** https://orcid.org/0000-0002-8249-4547 Content not peer-reviewed by ChemRxiv. **License:** CC BY-NC-ND 4.0

Molecular Formulas, and Substructure Searching. *Journal of Cheminformatics* **2017**, *9* (1), 33. https://doi.org/10.1186/s13321-017-0220-4.
(15) Zdrazil, B.; Felix, E.; Hunter, F.; Manners, E. J.; Blackshaw, J.; Corbett, S.; de Veij, M.; Ioannidis, H.; Lopez, D. M.; Mosquera, J. F.; Magarinos, M. P.; Bosc, N.; Arcila, R.; Kizilören, T.; Gaulton, A.; Bento, A. P.; Adasme, M. F.; Monecke, P.; Landrum, G. A.; Leach, A. R. The ChEMBL Database in 2023: A Drug Discovery Platform Spanning Multiple Bioactivity Data Types and Time Periods. *Nucleic Acids Research* **2024**, *52* (D1), D1180–D1192. https://doi.org/10.1093/nar/gkad1004.
(16) *ChEMBL Database*. https://www.ebi.ac.uk/chembl/ (accessed 2021-04-30).
(17) Weininger, D. SMILES, a Chemical Language and Information System. 1. Introduction to Methodology and Encoding Rules. *J. Chem. Inf. Model.* **1988**, *28* (1), 31–36. https://doi.org/10.1021/ci00057a005.
(18) Weininger, D.; Weininger, A.; Weininger, J. L. SMILES. 2. Algorithm for Generation of Unique SMILES Notation. *J. Chem. Inf. Comput. Sci.* **1989**, *29* (2), 97–101. https://doi.org/10.1021/ci00062a008.
(19) Bento, A. P.; Hersey, A.; Félix, E.; Landrum, G.; Gaulton, A.; Atkinson, F.; Bellis, L. J.; De Veij, M.; Leach, A. R. An Open Source Chemical Structure Curation Pipeline Using RDKit. *Journal of Cheminformatics* **2020**, *12* (1), 51. https://doi.org/10.1186/s13321-020-00456-1.

FUNDING SOURCES ACKNOWLEDGEMENT

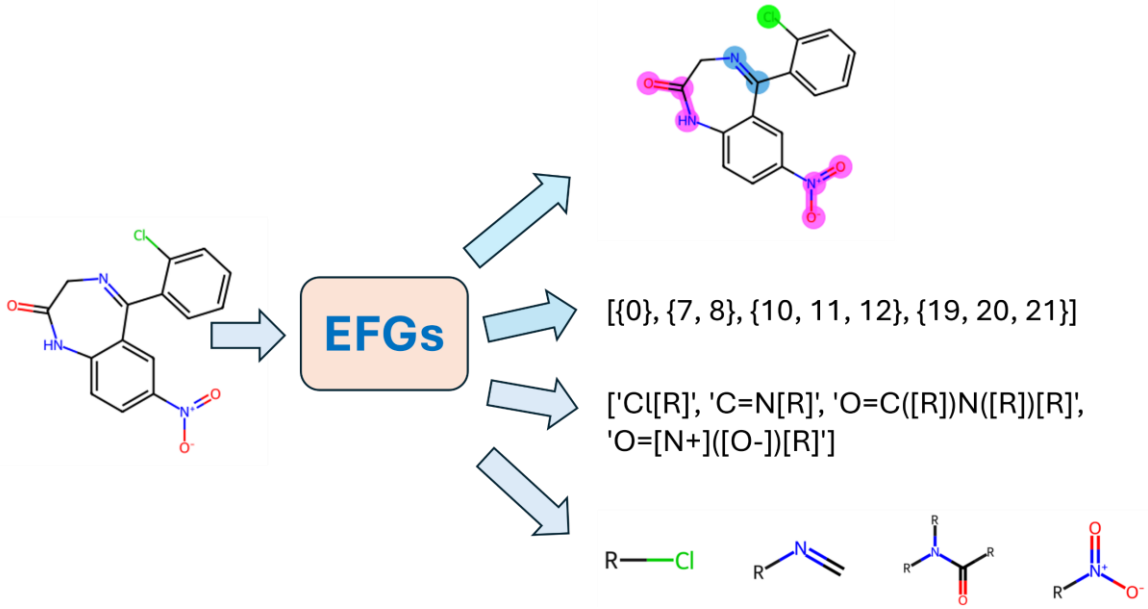
Grant PID2021-127318OB-I00 funded by MCIN/AEI/10.13039/501100011033 and by "ERDF A way of making Europe".

ACKNOWLEDGEMENTS

Dr. Peter Ertl is acknowledged for providing assistance in the correct interpretation of his algorithm. Dr. Greg Landrum is acknowledged for providing the RDKit toolkit freely available.

https://doi.org/10.26434/chemrxiv-2024-ptsbg **ORCID:** https://orcid.org/0000-0002-8249-4547 Content not peer-reviewed by ChemRxiv. **License:** CC BY-NC-ND 4.0

TABLE OF CONTENTS GRAPHIC



https://doi.org/10.26434/chemrxiv-2024-ptsbg **ORCID:** https://orcid.org/0000-0002-8249-4547 Content not peer-reviewed by ChemRxiv. **License:** CC BY-NC-ND 4.0